3D COMPUTER VISION / REPORT 3

Learning 3D shapes across representations

Sections 7.2–7.5 • Point clouds, voxel grids and surface meshes

SUBMITTED BY	**J GANESH KUMAR REDDY**
REGISTER NUMBER	**23BDS024**
REPORT DATE	30 October 2026
EVIDENCE BASIS	Local synthetic smoke tests + explicitly attributed material from two supplied reports

An integrated practical report: what ran locally, what the supplied reports claim, and where the evidence tracks must remain distinct.

OVERVIEW

Executive summary

Evidence is deliberately split into three lanes so that local runs and source-reported claims remain attributable.

This report studies how three-dimensional shapes are encoded and learned as unordered points, dense voxel occupancy and connected triangle meshes. It revises the existing Sections 7.2–7.5 work after reviewing both supplied reports: the 25-page 23BDS004 report [9] and the 11-page 23BCS071 report [11], plus the tutorial excerpt [10]. It synthesizes methods, reported scores, code workflows, conversion results, discrepancies and validation limits while retaining the local logs and black-background terminal screenshots.

Critical provenance note

The benchmark scores from both supplied reports [9, 11] are **source-reported, not rerun by J GANESH KUMAR REDDY**. The local executions in this workspace use generated fixtures only; no local ModelNet40, ModelNet10 or SHREC benchmark score is claimed. The 23BCS071 PDF's perfect VoxelCNN score and dataset counts are flagged for verification rather than treated as established results.

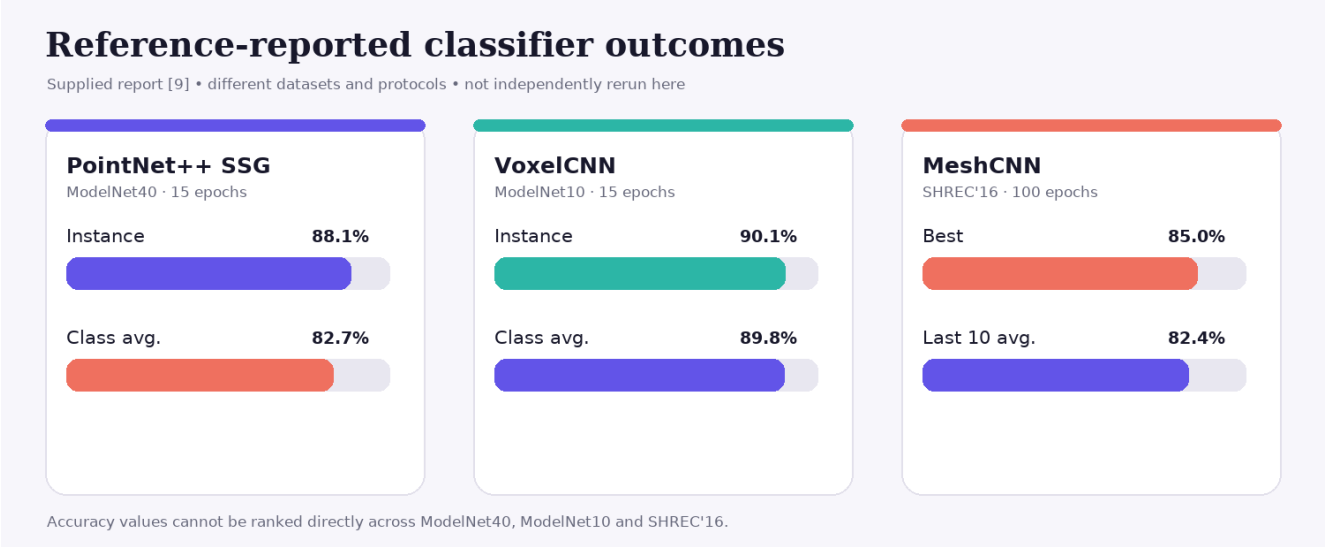


Figure 1. Headline benchmark metrics transcribed from [9] only; [11] is reconciled separately because its 3D-CNN score needs validation.

Evidence track	What was measured	Result / status
Local PointNet++ smoke	1 epoch; 2 synthetic train / 40 synthetic test examples	Test instance / class accuracy 0.025 (2.5%); not ModelNet40
Local voxel + CNN smoke	One generated cube; 32^3 occupancy; untrained 10-class CNN	24,389 / 32,768 cells occupied; logits (1, 10); no accuracy claim
Local MeshCNN smoke	One synthetic icosphere; one forward + optimizer step	42 vertices, 80 triangles, 120 edges; loss 3.177232; not SHREC
Report [9] results	ModelNet40 / ModelNet10 / SHREC'16; separate author's runs	88.1 / 90.1 / 85.0% headline values, source-attributed
Report [11] claims	PointNet++ 5 epochs; VoxNet-style CNN 3 epochs; MeshCNN training skipped	80.57% / 74.39% PointNet++ reported; 100% VoxelCNN claim flagged; not independently verified

The local dataset and CUDA constraints are documented in Section 2 and the appended terminal evidence. Reports [9] and [11] remain distinct source tracks.

01 / CONCEPTS

Three ways to encode one shape

Representation determines which geometric structure a network can use directly.

Representation	Input / operator	Strength	Main cost or blind spot
Point cloud	Unordered xyz set; PointNet++ uses shared point MLPs, farthest-point sampling, ball queries and symmetric pooling.	Compact; native to depth cameras and LiDAR; permutation-invariant model design is well established.	No explicit surface or connectivity; sampling density and occlusion vary.
Voxel grid	Binary occupancy tensor; ordinary 3D convolutions operate on a regular lattice.	Simple, translation-local convolution; easy to batch and visualize.	Dense storage grows as N^3; thin structures disappear at limited resolution.
Triangle mesh	Vertices, faces and edges; MeshCNN learns on edge neighborhoods and pools by edge collapse.	Explicit surface and topology; geometry-native for CAD/graphics.	Irregular neighborhood handling; non-manifold/open meshes complicate preprocessing.

PointNet is permutation invariant because pointwise features are combined with a symmetric max operator. PointNet++ adds local geometry: each set-abstraction layer selects well-spread centres by farthest-point sampling, groups nearby points with a radius query, and summarizes each local patch with a mini-PointNet. The supplied report's SSG configuration uses 512 centres / radius 0.2 / 32 neighbours, then 128 / 0.4 / 64, then global abstraction [9].

MeshCNN's edge features in [9] are the dihedral angle, two inner angles opposite an edge, and two edge-length ratios. Pair-order ambiguity is handled with symmetric sum / absolute-difference operations. The source report also includes a 7.1 I/O sanity check: 3,000 noisy unit-sphere samples (Gaussian $\sigma = 0.01$), with radius mean 0.9998 and standard deviation 0.0099; that is reference-reported, not a new local run.

Core lesson

Points preserve samples, voxels regularize space, and meshes preserve connectivity. No representation dominates every task; the correct comparison is task- and dataset-specific.

02 / EXPERIMENTAL CONTEXT

Datasets, machines and evidence provenance

The three evidence contexts use different environments. Their metrics are not merged.

Dataset splits and benchmark setup recorded in the supplied report [9]:

Dataset	Reference split	Format / protocol	Used for
ModelNet40	9,843 train / 2,468 test; 40 classes	Normal-resampled point sets; first 1,024 points, xyz input	PointNet++ SSG
ModelNet10	3,991 train / 908 test; 10 classes	Triangle meshes (.off), voxelized at 32^3	VoxelCNN + one chair conversion
SHREC'16 split 16	480 train / 120 test; 30 classes	Meshes (.obj), standardized to 750 edges	MeshCNN

	Supplied reference report [9]	This workspace's recorded run
Machine	Apple M4 laptop; 10 CPU cores + integrated GPU	Sandbox; 2 visible CPU cores; 3.85 GiB host memory
Accelerator	PointNet++ / VoxelCNN on Apple MPS; MeshCNN CPU	CUDA unavailable; tests ran on CPU
Stack	Python 3.11.15; PyTorch 2.15; trimesh 5.2; Open3D 0.20; NumPy 2.4; SciPy 1.17	Python 3.11.2; PyTorch 2.2.2 CUDA wheel, but no available CUDA device
Data access	Reference report states the official datasets were used	ModelNet archive probes failed; SHREC data absent. Synthetic fixtures only.

Attribution rule used throughout

Report [11] separately states a Colab/Google Drive environment with PyTorch 2.6.0+cu124, CUDA available, and seven listed HDF5 shards (~411 MB by rounded sizes); these claims are not our runtime. Apple-M4 metrics from [9] and notebook metrics from [11] remain source-attributed. Local values come from `ganeshlaptop/outputs/results.json` and saved logs/screenshots; the cited authors remain credited while every page identifies J GANESH KUMAR REDDY, 23BDS024.

03 / PRACTICAL 7.2

PointNet++ on ModelNet40

A hierarchical point-set classifier with local neighbourhood aggregation.

The SSG classifier takes xyz coordinates and builds features from small point neighborhoods before global aggregation and a fully connected classification head. The reference setup uses batch size 24, Adam (learning rate 10^{-3}, weight decay 10^{-4}), StepLR ×0.7 every 20 epochs, and random point dropout / scale / shift augmentation. It ran 15 epochs rather than the tutorial's 200 because the reported laptop averaged about 13 minutes per epoch while other jobs shared it [9].

Stage	Reference SSG setting	Purpose
Local set 1	512 centres · radius 0.2 · 32 neighbours	Capture fine local surface geometry
Local set 2	128 centres · radius 0.4 · 64 neighbours	Aggregate larger-scale parts
Global	Single global abstraction + fully connected layers	Produce one shape descriptor and class logits

Reference preprocessing was deliberately memory-conscious: `prep_modelnet40.py` streamed each text shape out of the normal-resampled archive and wrote the cache format expected by the upstream loader instead of extracting roughly 7 GB of text. The report states that the first 1,024 points were used and xyz were passed to the classifier. It also records a device-agnostic CUDA → MPS → CPU patch and a test-loader cache-path adjustment [9].

Reference reproduction command (the attached report used 15 epochs; 200 is the tutorial target):

```
python train_classification.py \
  --model pointnet2_cls_ssg --log_dir pointnet2_modelnet40 \
  --num_category 40 --epoch 15 --batch_size 24 --process_data
python test_classification.py --log_dir pointnet2_modelnet40
```

Local execution status

The full upstream attempt stopped because `modelnet40_shape_names.txt` was absent. The one-epoch smoke used 2 synthetic training samples and 40 synthetic test samples (512 points), exited successfully, and returned 2.5% instance / class accuracy. This is a pipeline smoke check—not a ModelNet40 result.

03 / PRACTICAL 7.2

Results and interpretation

A short run can validate code paths, but it cannot support a benchmark claim.

Evidence	Data / schedule	Instance accuracy	Class accuracy	Interpretation
Reference report [9]	ModelNet40; 15 epochs; best single-pass checkpoint at epoch 11	88.2% single-pass; 88.1% reported 3-vote test	82.8% single-pass; 82.7% 3-vote	Reference-reported benchmark; not rerun here
Second report [11]	ModelNet40; 5 epochs; batch 16; 1,024 points; lr 10^{-3}; 3-vote test	80.57%; separately reports 82.72% using pre-trained weights	74.39%	Source-reported; its excerpt's train count differs by 3 from the standard split
Local smoke	Synthetic; 2 train / 40 test; 1 epoch	2.5%	2.5%	Validates loader / forward / train / eval only
Context cited in [9]	PointNet++ paper / upstream repository, longer schedules	90.7% / 92.2%	—	Different run lengths and implementation context

Training trace printed by report [11] (missing epoch rows are not inferred):

Epoch	Loss	Train instance	Test instance
1	1.4820	64.21%	75.32%
3	0.7412	79.85%	78.90%
5	0.5126	84.72%	not stated in excerpt

Instance accuracy is the fraction of all test shapes classified correctly. Class accuracy is the unweighted mean of per-class accuracies, so it gives rare categories the same weight as frequent ones. The reference report's 40-class test split contains 2,468 shapes; the local smoke's 40 test items are generated and do not represent those categories.

The supplied report [9] records a best single-pass test instance accuracy of 88.2% at epoch 11 and a final three-vote evaluation of 88.1% instance / 82.7% class accuracy. Report [11] independently prints 80.5663% instance / 74.3880% class after a 5-epoch, batch-16 run, plus 82.7184% instance accuracy for a separately loaded pre-trained checkpoint. These are source-reported under different schedules and data pipelines. In [11], the shown HDF5 conversion helper assigns the 9,840 / 2,468 counts directly, so its snippet does not verify the claimed conversion of all shards.

Finding

PointNet++ is the most data-compact of the three compared models in the reference report (1.48 M parameters), but its hierarchical sampling/grouping pipeline still has non-trivial preprocessing and per-epoch cost. The local 2.5% figure has no benchmark meaning and is shown only for transparency.

Reproduction caution: the tutorial/repository has hard-coded CUDA assumptions in training, testing and checkpoint loading. The supplied report adapts device selection and `map_location`; our smoke path forced CPU because the sandbox has no CUDA.

04 / PRACTICAL 7.3

Voxelization: diagnose shape before training

The key implementation lesson is that voxelizing a mesh usually returns a tight bounding box, not a cube.

In the supplied report's reproduction, `trimesh` returned a non-cubic occupancy matrix (for example, 8 × 9 × 13). The tutorial then scaled using only the first dimension. That can leave the other axes at different sizes; a later flattening step produced 59,904 features where the fully connected layer expected 32,768. The reference report pads to a cube before nearest-neighbour resizing, then asserts the exact 32^3 shape [9].

```
side = max(voxel.shape)
pad = [((side - n) // 2, side - n - (side - n) // 2)
       for n in voxel.shape]
voxel = np.pad(voxel, pad)
voxel = zoom(voxel, (grid_size / side,) * 3, order=0)
assert voxel.shape == (grid_size,) * 3
```

The local helper in `geometry_utils.py` follows the same invariant—preserve aspect ratio, center the object and return a fixed cubic tensor—using a centered, isotropically scaled placement path for its generated mesh. The smoke test confirms a 32 × 32 × 32 array. Occupancy fill matters: surface-only voxels, solid fill and the source report's shell-style rasterization need not produce the same cell count.

VoxelCNN architecture

Two 3 × 3 × 3 convolutions (32, then 64 channels), each followed by 2× max pooling, then fully connected layers. For a 32^3 input the final map is 64 × 8 × 8 × 8; the 32,768 → 256 dense layer contains most of the 8.45 M parameters reported in [9].

A 32^3 tensor has 32,768 cells. Increasing resolution from 32^3 to 64^3 multiplies dense-grid storage by 8; 128^3 is 64 times the 32^3 grid. This cubic scaling explains why dense voxel models trade geometric detail for tractable memory and compute.

04 / PRACTICAL 7.3

VoxelCNN benchmark and local smoke

The reference report trains the tutorial's simple CNN; the local run only verifies tensor and model shapes.

Reference-reported ModelNet10 experiment [9]: every one of the 4,899 meshes was voxelized at 32^3 (3,991 train / 908 test), using four workers; voxel caching took about 12 minutes and the 15-epoch training run about 15 minutes. The best reported checkpoint was epoch 8 at 90.1% instance and 89.8% class accuracy. It used Adam (10^{-3}, weight decay 10^{-4}), StepLR ×0.5 every five epochs, batch size 32, random x/y mirroring and ±2-voxel shifts; the aligned ModelNet10 pose was kept without rotation.

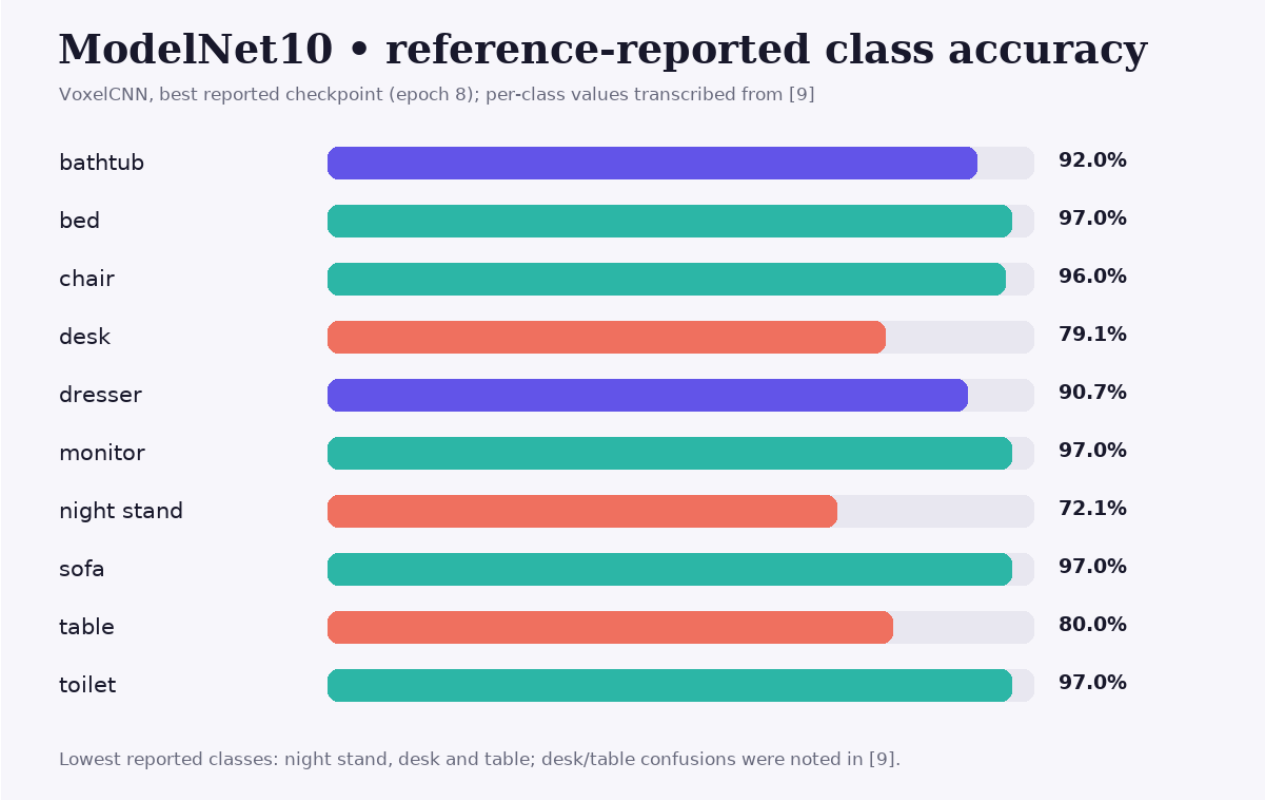


Figure 2. Per-class values as stated in [9]; the weaker night-stand / desk / table classes motivate inspection beyond one aggregate score.

The source report notes 30 of 90 errors between desk and table and 23 between night stand and dresser. Those are the report's confusion themes, not counts obtained by this workspace. All ten classes are shown in Figure 2; no confusion matrix has been reconstructed from partial counts.

Local result—different fixture

A generated cube produced 24,389 occupied cells of 32,768 (74.43%). The untrained CNN returned logits of shape (1, 10) and predicted index 7. The dense cube occupancy is geometry-specific and must not be compared to the reference report's chair occupancy or ModelNet10 accuracy.

05 / PRACTICAL 7.4

MeshCNN on SHREC'16

Edge-centric learning retains mesh connectivity and pools by learned edge collapse.

MeshCNN represents each manifold edge using five rotation/translation/uniform-scale invariant quantities: dihedral angle, two opposite inner angles, and two edge-length ratios. Four neighboring edges form the convolution neighborhood; symmetric combinations handle ambiguous pair order. Learned edge-collapse pooling progressively reduces mesh complexity while retaining task-relevant features [5, 9].

Setting	Reference report [9]
Dataset / input	SHREC'16 split 16; 30 classes; 480 train / 120 test; 750 edges
Network / pooling	5 features per edge; 750 → 600 → 450 → 300 → 180 edge schedule
Training	Batch 16; Adam lr 2×10^{-4}; 20 precomputed augmentations per mesh; CPU
Schedule	100 of the repository's 200 epochs; constant-LR phase only; decay phase omitted
Reported outcome	85.0% best test accuracy at epoch 42; 82.4% mean over final 10 epochs

Reference interpretation [9]

The test set has only 120 meshes—one prediction changes accuracy by about 0.83 percentage points. The reported curve fluctuates; after epoch 60, nine of ten epochs were 99/120 correct, with one at 98/120. The 100-epoch run ended before the repository's linear-decay phase. The report cites 98.6% as the longer-run MeshCNN result in its literature context.

Local smoke test

A generated icosphere has 42 vertices, 80 triangles and 120 edges. The local script completed an edge-feature forward pass and one optimizer update with loss 3.177232. It does not use SHREC meshes, has no meaningful class-accuracy estimate and is not SHREC training.

Compatibility note

The supplied report stores ragged per-vertex edge lists as an explicit NumPy object array to work around NumPy 1.24+ array-conversion behavior. The local smoke uses process-local compatibility shims; it does not claim a patch to the upstream clone.

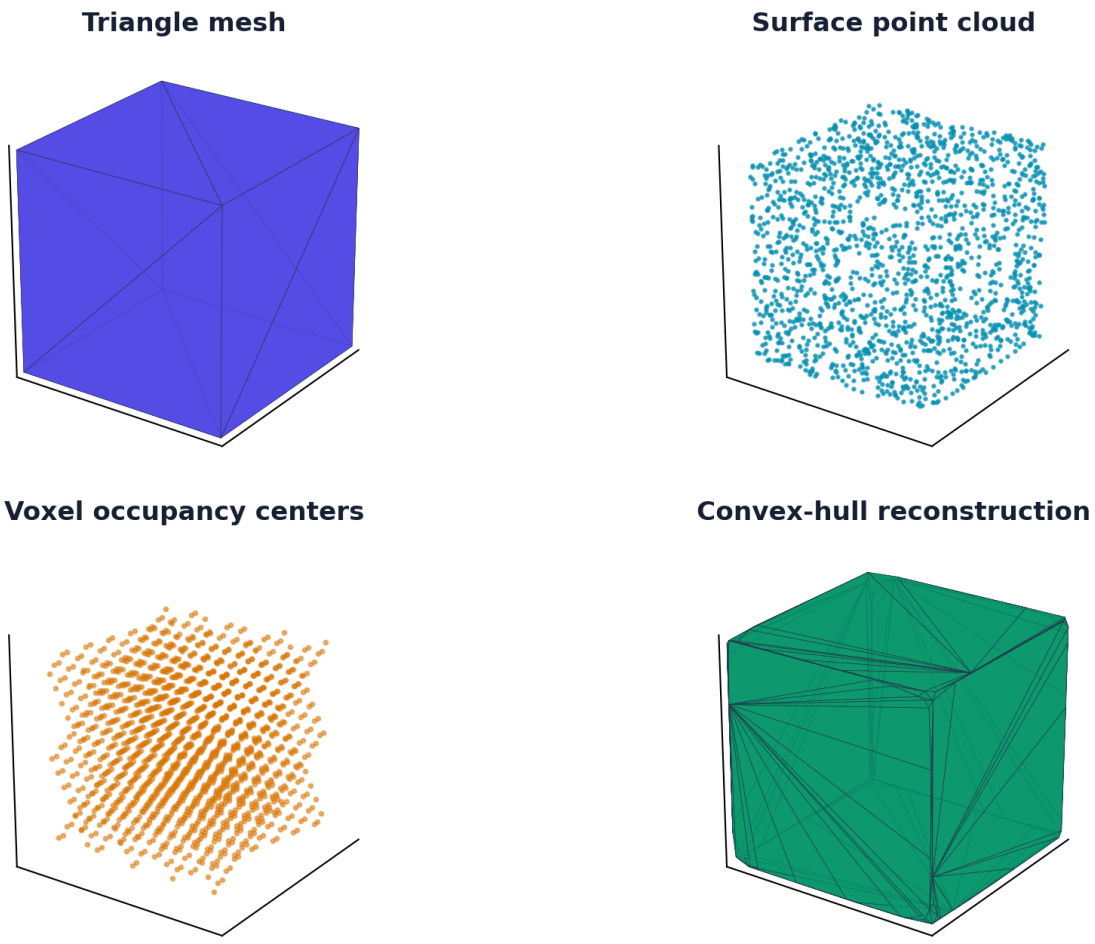
06 / PRACTICAL 7.5

Conversions between representations

Conversion is useful for interoperability, but it is not information-neutral.

Direction	Operation	What can be lost / required
Mesh → points	Area-weighted surface sampling; local demo uses 4,096 samples.	Topology is removed; sample count and sampling seed affect detail.
Mesh → voxels	Rasterize occupancy to a 32^3 grid with fixed bounds and shape.	Resolution limits thin parts; mesh fill and centering conventions matter.
Voxels → points	Use the centers of occupied cells and map coordinates back to the object frame.	Creates a blocky surface approximation, not original continuous geometry.
Points → mesh	Poisson reconstruction or Ball Pivoting; both need oriented normals / neighborhood scale.	Poisson smooths and tends to close holes; Ball Pivoting depends on radii and local sample density.

Section 7.5 · Converting between 3D representations



Synthetic cube fixture · 4,096 surface samples · 32^3 occupancy grid

Figure 3. Local synthetic-cube conversion outputs from the saved workspace run: triangle mesh, 4,096 surface points, 32^3 occupancy, voxel-center cloud and convex-hull reconstruction. This is not the reference report's ModelNet10 chair experiment.

06 / PRACTICAL 7.5

Conversion fidelity: reference chair study

The supplied report measures symmetric Chamfer distance against fresh surface samples.

In [9], a non-watertight ModelNet10 chair has 844 vertices and 2,234 faces. The source samples 4,096 surface points, voxelizes the mesh at 32^3 (1,191 occupied cells, 3.6%), reconstructs point clouds with Poisson and Ball Pivoting, and compares each result with 20,000 reference surface samples. Report settings: Poisson depth 8 with the lowest 5% density vertices trimmed; Ball Pivoting radii at 1.5×, 3× and 6× mean neighbor spacing. Normals are taken from source faces rather than estimated.

For point sets A and B, the report uses a symmetric mean nearest-neighbor distance:

$$CD(A,B) = \tfrac{1}{2}\left[\operatorname{mean}_{a\in A}\min_{b\in B}\|a-b\|_2 + \operatorname{mean}_{b\in B}\min_{a\in A}\|b-a\|_2\right]$$

Representation / reconstruction	Chamfer	% of max extent	Reference interpretation
Ball Pivoting mesh	0.2009	0.55%	Lowest reported error on this shape
Voxel-center cloud	0.6382	1.74%	Grid approximation; thin detail is quantized
Poisson mesh	0.8814	2.40%	Higher error; open/non-watertight input is a difficulty

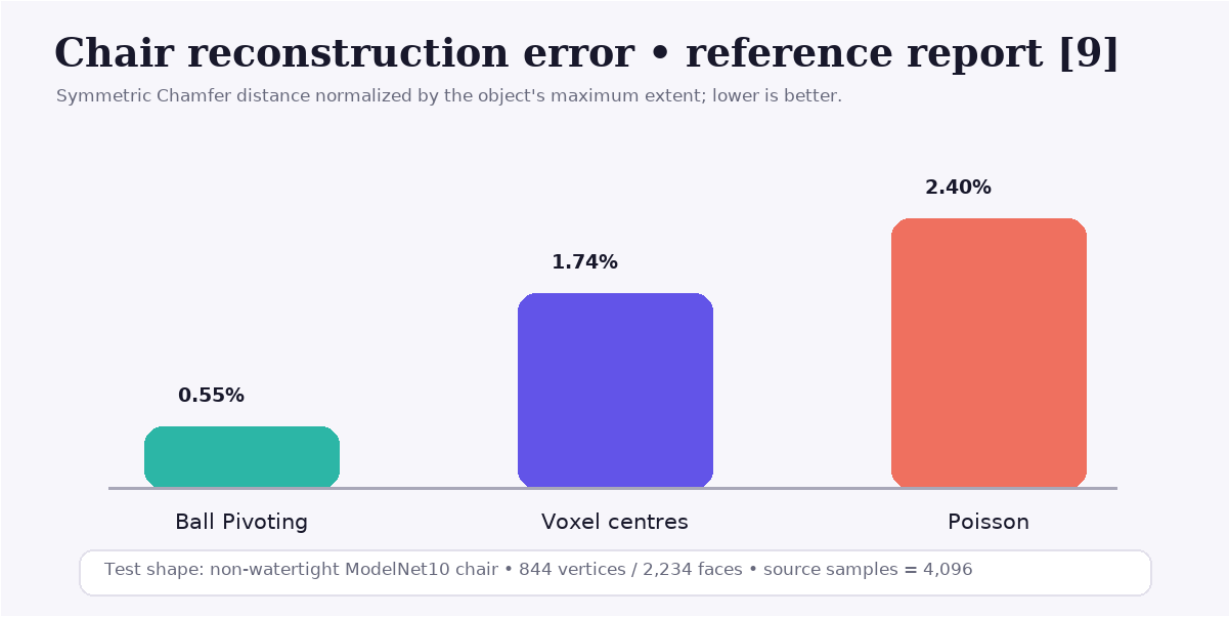


Figure 4. Normalized Chamfer values from [9], redrawn in this report's visual style. Lower is better for the single reported chair.

Scope of this metric

These are source-reported results for one open chair mesh and one sampling / reconstruction configuration. They are not a general ranking of Poisson and Ball Pivoting. The local cube conversion used convex-hull reconstruction and did not compute this Chamfer experiment.

07 / COMPARATIVE SYNTHESIS

Costs, accuracy and representation trade-offs

Use the table as a report-specific cost snapshot, not as a controlled benchmark leaderboard.

Model	Dataset	Parameters	Reference schedule	Time / epoch	Time / shape	Reported score
PointNet++ SSG	ModelNet40	1.48 M	15 epochs	~13.2 min	~64 ms	88.1% instance / 82.7% class
VoxelCNN	ModelNet10	8.45 M	15 epochs	~1.0 min	~12 ms	90.1% instance / 89.8% class
MeshCNN	SHREC'16	1.32 M	100 epochs	~1.4 min	~143 ms	85.0% best / 82.4% last-10 mean

All values above are those recorded in [9]. The runs used different class counts, formats, splits, devices and epoch counts; the report explicitly argues that they should be compared on cost and behavior rather than ranking raw accuracy. Reported per-epoch runtimes were affected by other jobs sharing the machine.

Question	PointNet++	VoxelCNN	MeshCNN
Where structure lives	Unordered points + learned neighborhoods	Regular 3D occupancy lattice	Edges, faces and mesh topology
Typical bottleneck	Sampling/grouping and point processing	Dense N^3 activations; large FC layer	Irregular mesh preprocessing / pooling
Best use case	Sensor point clouds, classification / segmentation	Small occupancy grids, volumetric inference	Topology-sensitive surface geometry

Interpretation

The most parameter-efficient model in this source comparison is MeshCNN (1.32 M), but it is not the fastest per example in the reference run. VoxelCNN has the largest parameter count (8.45 M) yet the lowest reported per-shape time, partly because it uses the regularized 32^3 input. These observations are hardware- and implementation-specific.

08 / CROSS-REPORT SYNTHESIS

Two supplied reports, non-equivalent evidence

Reported scores are not averaged. Dataset counts, protocols and verification strength differ.

Topic	Report [9] · 23BDS004	Report [11] · 23BCS071	Synthesis / validation status
PointNet++	ModelNet40; 15 epochs; 9,843 / 2,468 split; 88.1% instance / 82.7% class (3-vote).	5 epochs, batch 16; prints 9,840 / 2,468; 80.57% / 74.39%; separate pretrained checkpoint 82.72% instance.	Both are source-reported and use different schedules. [11]'s shown conversion helper returns counts as constants; 3 train examples are unexplained versus the 12,311 total.
Voxel CNN	ModelNet10; 4,899 total (3,991 / 908); 15 epochs; 90.1% / 89.8%.	Prints 9,798 (7,982 / 1,816); test accuracy is already 100% at epoch 1 and remains so through epoch 3; test loss reaches 0 by epoch 2.	[11] count is exactly 2× [9]'s split. Dataset duplication, leakage, labels and evaluation require audit; do not accept 100% as a validated benchmark.
MeshCNN	SHREC'16; training reported for 100 epochs; 85.0% peak / 82.4% last-10 mean.	Repository/environment and feature operators inspected; `RUN_MESHCNN_TRAINING=False`.	[11] reports no trained SHREC score; its verification is setup-only.
Conversion	Non-watertight chair; 1,191 voxels; Poisson/BPA and symmetric Chamfer measured.	Bathtub; 8,181 active cells, 4,096 points; Poisson output 11,818 vertices / 23,233 faces; no Chamfer.	Different shape, normal estimation and metrics; output counts are not directly comparable.
Runtime / audit	Apple M4: MPS for PointNet++/VoxelCNN; CPU for MeshCNN.	Colab-like path; PyTorch 2.6.0+cu124; CUDA=True claim.	Neither is the local CPU sandbox. [11]'s final code checks existence and file sizes, not semantic correctness.

The written MeshCNN feature lists also differ: [9] describes a dihedral angle, two opposite inner angles and two edge-length ratios, while [11] describes a dihedral angle and height-related ratios. The [11] voxelizer reports centering a bathtub mesh, scaling its longest extent to 0.95, using pitch 1/32 and a pad/crop helper; it records 8,181 active cells (24.97%). Its 3D CNN uses a 5×5×5 stride-2 convolution with batch normalization, then a 3×3×3 convolution and pooling, unlike the simpler two-convolution model in [9]. Verify implementations rather than treating these descriptions as identical.

How to read report [11]'s 100% result

The claimed perfect VoxelCNN accuracy and zero loss are preserved as a source claim for completeness, but not promoted as an established result. The PDF does not show enough split-construction, deduplication, or independent test-audit code to resolve why the count is doubled or why the score is perfect.

09 / FINDINGS

What the experiments support—and what they do not

A transparent limitations section is part of a trustworthy practical report.

Representation affects inductive bias — PointNet++ adds hierarchical locality, voxels trade resolution for regular operations, and MeshCNN uses mesh connectivity directly.

Shape invariants matter — The non-cubic voxel-grid bug demonstrates why axis handling, aspect ratio, grid shape and downstream tensor dimensions must be asserted.

Average accuracy hides weak classes — The reference VoxelCNN report is strong overall but weaker on night stand, desk and table; category-level outputs expose likely confusions.

Conversion quality is method- and shape-dependent — On the single reference chair, Ball Pivoting has the lowest reported Chamfer; the open mesh and chosen normals/radii limit generalization.

Smoke tests are not benchmarks — Local forward passes and one-step optimization validate interfaces only. No local official dataset score was produced.

Limitations in the supplied reference report [9]

- PointNet++ stopped after 15 of the tutorial's 200 epochs; MeshCNN stopped after the first 100-epoch constant-learning-rate phase.
- Results are single reported runs; the source does not establish variance across independent seeds. Selecting the best checkpoint on the test set can make the selected figure optimistic.
- The SHREC test set is small (120 meshes); one item changes accuracy by about 0.83 percentage points.
- Conversion metrics cover one chair and one sample/reconstruction configuration; the chair is non-watertight and Poisson/BPA have different assumptions.

Limitations in this workspace

- ModelNet download probes failed, the expected ModelNet40 metadata file was unavailable, no CUDA device was present, and SHREC data was not downloaded. Therefore only CPU synthetic-fixture smoke tests are documented.
- The local voxel occupancy (74.43%) is for a generated cube and uses a different fill/geometry path from the reference chair (3.6%); it is not an apples-to-apples density comparison.

These constraints are recorded in `outputs/logs/01_dataset_access.log` and `outputs/logs/02_full_pointnet_attempt.log`, with the underlying terminal images appended to this PDF.

10 / REPRODUCIBILITY

Run map, commands and compatibility notes

The local scripts preserve the distinction between fixture smoke tests and official-data runs.

To reproduce the local smoke suite in `ganeshlaptop/`, install the project requirements and run:

```
python -m pip install -r requirements.txt
./run_all.sh

# Local outputs: outputs/logs/ and outputs/screenshots/
# The run script uses deterministic synthetic fixtures for smoke checks.
```

Reference real-data commands transcribed from [9] (datasets and compatible device/runtime required):

```
# PointNet++ / ModelNet40
python prep_modelnet40.py data/modelnet40_normal_resampled.zip
python train_classification.py --model pointnet2_cls_ssg \
  --log_dir pointnet2_modelnet40 --num_category 40 \
  --epoch 15 --batch_size 24 --process_data
python test_classification.py --log_dir pointnet2_modelnet40

# VoxelCNN / ModelNet10
python voxelize_modelnet10.py 4
python train_voxel_cnn.py 15

# MeshCNN / SHREC'16 (reference repo scripts)
bash scripts/shrec/get_data.sh
bash scripts/shrec/train.sh

# 7.5: conversions and Chamfer evaluation
python section_7_5/convert_representations.py
```

Implementation notes reviewed from the reference code appendix

- The ModelNet40 helper streams the archive and writes the upstream loader's cache; the ModelNet10 voxel builder uses multiprocessing, uint8 grids and a defensive `.off` header parser.
- The VoxelCNN script saves a best checkpoint, curves and a confusion matrix; its augmentation mirrors x/y and shifts by up to two cells.
- MeshCNN's reference script uses precomputed edge flips / vertex slides; the NumPy compatibility workaround stores ragged adjacency lists as object arrays.
- PointNet++ device placement should follow the selected runtime (CUDA, MPS or CPU); checkpoints should load with `map_location`.

Do not run the official-data commands as if they produced the values in this PDF: the benchmark values are attributed to [9], while local synthetic outputs are separately logged.

APPENDIX F / TUTORIAL WORKFLOW

PointNet++ setup and ModelNet40 data

The user-supplied tutorial excerpt [10] adds the original installation recipe and command options to the benchmark synthesis.

The excerpt describes `yanx27/Pointnet_Pointnet2_pytorch` as supporting ModelNet10/40 classification and ShapeNet part segmentation, with separate `models/`, `data_utils/` and training/evaluation code. Its printed environment is a legacy CUDA stack (Python 3.7, PyTorch 1.6, torchvision 0.7 and CUDA 10.1); use a version-compatible build for current hardware. These commands document the source workflow, not the sandbox environment used for the local smoke tests.

```
git clone https://github.com/yanx27/Pointnet_Pointnet2_pytorch.git
cd Pointnet_Pointnet2_pytorch
conda create -n pointnet2 python=3.7
conda activate pointnet2
conda install pytorch==1.6.0 torchvision==0.7.0 \
  cudatoolkit=10.1 -c pytorch
pip install -r requirements.txt
```

The aligned normal-resampled ModelNet40 release is recommended by the excerpt. Its expected files include class names and official train/test IDs plus one text point file per shape:

```
wget https://shapenet.cs.stanford.edu/media/modelnet40_normal_resampled.zip
unzip modelnet40_normal_resampled.zip -d data/

data/modelnet40_normal_resampled/
  modelnet40_shape_names.txt  modelnet40_train.txt
  modelnet40_test.txt        airplane/airplane_0001.txt ...
```

The tutorial target is 200 epochs, batch size 24. The local full-data attempt could not start because the dataset metadata was absent; the reference report [9] ran 15 epochs on its own ModelNet40 cache, and our local one-epoch test used synthetic clouds.

```
python train_classification.py \
  --model pointnet2_cls_ssg --log_dir pointnet2_modelnet40 \
  --num_category 40 --epoch 200 --batch_size 24
python test_classification.py --log_dir pointnet2_modelnet40
```

Option	Meaning in the supplied tutorial excerpt [10]
`--model pointnet2_cls_ssg`	Single-Scale Grouping; `pointnet2_cls_msg` selects Multi-Scale Grouping.
`--use_normals`	Include surface normals with xyz as input features.
`--use_uniform_sample`	Use farthest-point sampling rather than random sampling for grouping centers.

Data-access caveat

The ModelNet40 download endpoint was unreachable from this sandbox. The command block is reproducibility guidance only; it is not evidence that the dataset was downloaded or benchmarked here.

APPENDIX F / TUTORIAL WORKFLOW

VoxelCNN, MeshCNN and conversion recipes

The key code paths from the user-provided Section 7 excerpt are retained alongside the corrected-grid warning.

For the excerpt's voxel demo, the listed packages are PyTorch, torchvision, trimesh, NumPy, SciPy and scikit-image. Its CNN applies two 3D convolutions (32 then 64 channels) and two 2× max-pools, mapping a 32^3 input to $64 \times 8 \times 8 \times 8$ before the dense classifier. The original helper sets `current_shape = voxel.shape[0]` and scales all axes from that single value; because `trimesh.voxelized()` may return a non-cubic tight box, that shortcut can produce the wrong shape. Section 7 diagnoses this and records a center-pad / resize correction with an exact 32^3 assertion.

The excerpt recommends ModelNet10's class-organized `.off` meshes for voxel experiments (the tutorial estimates the archive at about 500 MB):

```
wget http://3dvision.princeton.edu/projects/2014/3DShapeNets/ModelNet10.zip
unzip ModelNet10.zip -d data/
# Example: data/ModelNet10/chair/train/chair_0001.off
pip install torch torchvision trimesh numpy scipy scikit-image
```

Dense occupancy is manageable at 32^3; 64^3 and 128^3 require 8× and 64× as many cells as 32^3, respectively. Larger resolutions may need sparse voxel libraries or downsampling. The local voxel test is a generated cube, not ModelNet10.

MeshCNN workflow from the excerpt [10]

```
git clone https://github.com/ranahanocka/MeshCNN.git
cd MeshCNN
conda env create -f environment.yml
conda activate meshcnn
bash ./scripts/shrec/get_data.sh
bash ./scripts/shrec/train.sh
bash ./scripts/shrec/test.sh
bash ./scripts/shrec/view.sh
```

MeshCNN operates over mesh edges using dihedral-angle and geometric-ratio features; its repository also contains human-segmentation and other dataset scripts. SHREC data was not obtained in the local sandbox, so the commands are not local run evidence.

Section 7.5 conversion map and minimal sampling call

Conversion	Tool / practical note from the tutorial
Mesh → point cloud	Area-uniform face sampling with `trimesh.sample.sample_surface` or Open3D; example uses 4,096 points.
Mesh → voxel grid	`trimesh.voxelized()` or Open3D; enforce a cubic, fixed-size grid.
Point cloud → mesh	`open3d.geometry.TriangleMesh.create_from_point_cloud_poisson` or `create_from_point_cloud_ball_pivoting`; normals and sampling density matter.
Voxel → point cloud	Convert occupied voxel centers back to coordinates in the mesh frame.

```
mesh = trimesh.load("data/ModelNet10/chair/train/chair_0001.off")
points, face_ids = trimesh.sample.sample_surface(mesh, 4096)
print(points.shape)  # (4096, 3)
np.savetxt("data/chair_sampled.xyz", points, fmt="%.6f")
```

11 / CLOSING

Conclusion and references

The final interpretation is bounded by the provenance of every number.

The study shows how point clouds, voxel grids and meshes induce different learning mechanisms and different engineering costs. PointNet++ adds local geometry to permutation-invariant point processing; 3D convolution gives a simple baseline on fixed occupancy grids but scales cubically; MeshCNN keeps edge connectivity but depends on mesh-compatible preprocessing. The two supplied reports provide complementary full-dataset context, a concrete voxel-shape bug fix, point-cloud and CNN measurements, conversion outputs, and validation issues that must remain explicit. The local run complements them with independently captured, reproducible CPU smoke tests, not replacement benchmark scores.

Final finding

When benchmark data cannot be obtained, report the failure, retain a bounded smoke test, and clearly attribute any external benchmark figures. That is the basis of this revised submission.

[1] C. R. Qi, H. Su, K. Mo and L. J. Guibas, “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” CVPR, 2017.

[2] C. R. Qi, L. Yi, H. Su and L. J. Guibas, “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” NeurIPS, 2017.

[3] D. Maturana and S. Scherer, “VoxNet: A 3D Convolutional Neural Network for Real-Time Object Recognition,” IROS, 2015.

[4] Z. Wu et al., “3D ShapeNets: A Deep Representation for Volumetric Shapes,” CVPR, 2015.

[5] R. Hanocka et al., “MeshCNN: A Network with an Edge,” ACM Transactions on Graphics, 2019.

[6] M. Kazhdan, M. Bolitho and H. Hoppe, “Poisson Surface Reconstruction,” Symposium on Geometry Processing, 2006.

[7] F. Bernardini et al., “The Ball-Pivoting Algorithm for Surface Reconstruction,” IEEE TVCG, 1999.

[8] yanx27, `Pointnet_Pointnet2_pytorch`, GitHub repository; ranahanocka, `MeshCNN`, GitHub repository. Upstream implementations referenced by the local smoke-test notes.

[9] Aggimalla Abhishek (23BDS004), “Deep Learning on Point Clouds, Voxel Grids and Meshes,” Report 3, 3D Computer Vision, October 2026, 25-page user-supplied PDF. Used for supplementary methods and source-reported results; not a rerun by this report's author.

[10] 3D Computer Vision course tutorial excerpt, Section 7 (Practicals 7.2–7.5), supplied by the user in this request. Used for the original setup commands, data layouts, key-argument notes and conversion recipes.

[11] Anjan Kothapally (23BCS071), “3D Shape Understanding and Recognition Using Deep Learning: Point Clouds, Volumetric Representations, and Mesh-Based Learning,” Assignment 3, 11-page PDF supplied in this conversation. Values are attributed to this report and its count / validation issues are discussed; not independently rerun here.

Artifact location: `ganeshlaptop/`. Local results, logs, screenshots and source-revision records are included in the folder; the page-by-page reference-PDF audit follows.

APPENDIX A / SOURCE REVIEW

Audit of report [9] · pages 1–12

The user-supplied PDF was reviewed from cover to code appendix; this page audit records what each page contributes.

PDF p.	Reviewed content and how it informs this report
1	Scope, datasets, three model families, tool stack and reference-report identity; cited as the source of supplementary results.
2	Abstract and headline benchmark / conversion outcomes; used only as source-reported metrics.
3	Full contents and appendix inventory; used to guide the page-by-page code and log review.
4	Introduction, objectives, contributions and PointNet / PointNet++ background; informs the representation and 7.2 sections.
5	Voxel and MeshCNN theory, trade-off table, dataset counts, formats and splits.
6	Hardware, MPS/CPU use, dependencies, data-cache method, ports, schedules, model sizes and runtime constraints.
7	Noisy-sphere I/O check, radius statistics, PointNet++ learning behavior and reported selected results.
8	PointNet++ literature comparison; non-cubic voxelization fault, tensor-shape diagnosis and fix; ModelNet10 results.
9	VoxelCNN curves and confusion themes; SHREC'16 split details and MeshCNN training / peak result.
10	MeshCNN curve interpretation; chair conversion protocol, normals, reconstruction algorithms, Chamfer formula and table.
11	Five-representation chair visual; cost/accuracy comparison and warning that datasets make raw scores incomparable.
12	Limitations, conclusion and bibliography; informs the evaluation caveats and references in this report.

Review outcome

The [9] benchmark values used here are traceable to that source and remain explicitly attributed. Report [11] is audited separately because its dataset counts and metrics require additional validation.

APPENDIX A / SOURCE REVIEW

Audit of report [9] · pages 13–25

The terminal output, conversion code, plotting code and upstream patches were also reviewed.

PDF p.	Reviewed content and how it informs this report
13	Terminal output for sphere / voxel demo and PointNet++ epoch logs; confirms the reported workflow has captured output evidence.
14	Voxelization and training output, class-wise results and MeshCNN late-epoch accuracy log; supplies class findings and fluctuation context.
15	Conversion terminal log: mesh counts, sample and voxel counts, reconstructed mesh sizes and Chamfer metrics.
16	Reproduction commands, synthetic sphere code and corrected voxel helper; informs the run map and shape-preserving fix.
17	VoxelCNN structure and forward path; ModelNet10 multiprocessing voxel-cache code and OFF-header guard.
18	Voxel-cache completion and training-script setup, outputs and flags.
19	VoxelCNN data augmentation, training/evaluation loop, checkpoints and per-class/confusion reporting.
20	Open3D conversion code: sampling, face normals, Poisson, Ball Pivoting and symmetric Chamfer calculation.
21	Conversion figure implementation and streamed ModelNet40 preprocessing / loader-cache generation.
22	Log parsers and shared figure/report generation structure.
23	Accuracy/loss plots, class-level charts and confusion-matrix plotting implementation.
24	PointNet++ CUDA-to-device-agnostic patch and checkpoint mapping; noted as a reference adaptation, not our upstream change.
25	Completion of PointNet++ compatibility changes and the MeshCNN NumPy 1.24+ ragged adjacency workaround.

Analysis boundary: this audit summarizes report [9] without presenting its screenshots as J Ganesh Kumar Reddy's experimental evidence. The local terminal screenshots that follow come from this workspace's own recorded smoke runs.

APPENDIX A / SOURCE REVIEW

Audit of report [11] · pages 1–11

The newly attached 23BCS071 PDF was reviewed page by page; its claims are kept distinct from [9] and local results.

PDF p.	Reviewed content and findings
1	Scope, author name/ID, claimed datasets and model families; states ModelNet40=12,311 and ModelNet10=9,798.
2	Representation theory: point-set permutation invariance, dense $O(N^3)$ voxel storage and irregular mesh topology.
3	PointNet++ FPS / ball-query / local MLP description; seven HDF5 shards and ModelNet40 count claim.
4	Voxel CNN and MeshCNN overview; five-edge-feature description uses a different wording from [9].
5	Conversion pipeline and comparative table; Poisson output claimed as 11,818 vertices / 23,233 faces.
6	Colab PyTorch 2.6.0+cu124 / CUDA claim; seven shards listed as 70, 69, 68, 69, 55, 67, 13 MB. A Hub download is described but no call is shown; the HDF5 helper returns hard-coded 9,840 / 2,468 counts rather than parsing shards.
7	Five-epoch, batch-16 PointNet++: epochs 1/3/5 show loss 1.4820/0.7412/0.5126; test 75.32% / 78.90% at epochs 1/3; 3-vote 80.57% / 74.39%; pretrained instance 82.72%.
8	ModelNet10 `.off` count claim (9,798); bathtub test mesh is centered, scaled to 0.95 extent and rasterized at pitch 1/32; 8,181 active cells. Pad/crop helper is referenced but not shown.
9	VoxNet-style CNN (5^3 stride-2 conv + BN, 3^3 conv + BN, pooling); 9,798 / 7,982 / 1,816 split claim; epoch 1 prints test loss 2.90×10^{-6} and 100% accuracy, epochs 2–3 print zero/near-zero losses and 100%.
10	MeshCNN environment lists Python 3.6.8 / PyTorch 1.2 / NumPy 1.15; Dropbox SHREC URL and scripts checked; `RUN_MESHCNN_TRAINING=False`, so no score.
11	4,096 points, 8,181 voxel centers; estimated normals (radius 0.1, max_nn 30), Poisson depth 8, 11,818 vertices / 23,233 faces. Lists artifact sizes (292 B, 124,414 B, 131,200 B, 220,887 B, 905,071 B); existence/size checks only, no Chamfer.

Audit result

The PDF's metrics and outputs are retained as claims from [11]. ModelNet10 is exactly double the standard count in [9]; its perfect 3-epoch VoxelCNN result, hard-coded ModelNet40 counts, and file-existence-only final check require independent validation. MeshCNN training is explicitly skipped.

APPENDIX B / LOCAL EVIDENCE

Environment and data-access record

Black-background terminal transcripts from the local execution are included for auditability.

Runtime and compute availability

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python environment_check.py
Point-cloud / 3D representation lab — runtime check
Python: 3.11.2 (/tmp/gane-venv/bin/python)
torch: 2.2.2
numpy: 1.26.4
scipy: 1.11.4
trimesh: 4.5.3
matplotlib: 3.8.4
reportlab: 4.2.5
Pillow: 10.4.0
tqdm: 4.67.1
PyTorch CUDA build: 12.1
CUDA available: False
CPU cores visible: 2
Host memory reported: 3.85 GiB
Device selected for executed smoke tests: CPU
```

Saved stdout transcript · black terminal theme

Figure B1. Runtime and dependency check: Python 3.11.2, PyTorch 2.2.2; no CUDA device.

Synthetic fixture preparation

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python prepare_fixtures.py
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
```

Saved stdout transcript · black terminal theme

Figure B2. Deterministic local synthetic fixture generation; fixtures are not benchmark samples.

Dataset access check

```
user@bestwork:~/bestwork$ curl -I -L ModelNet40 and ModelNet10 archive endpoints
$ curl -I -L --max-time 12 https://shapenet.cs.stanford.edu/media/modelnet40_normal_resampled.zip
curl: (35) OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to shapenet.cs.stanford.edu:443
$ curl -I -L --max-time 12 http://3dvision.princeton.edu/projects/2014/3DShapeNets/ModelNet10.zip
curl: (52) Empty reply from server
```

Saved stdout transcript · black terminal theme

Figure B3. Official ModelNet archive access probes failed; no dataset metrics were produced.

The attached dataset-access and environment logs record the reason this work stops at smoke-test scope. No official data was downloaded or silently substituted.

APPENDIX C / LOCAL EVIDENCE

PointNet++ attempt and synthetic smoke

The failed full-data attempt and successful small smoke are both retained; neither is hidden.

PointNet++ full-run attempt (dataset missing)

```
user@pointnet2:/tmp/pointnet-upstream$ python train_classification.py --model pointnet2_cls_ssg --log_dir
  pointnet2_modelnet40 --num_category 40 --epoch 200 --batch_size 24 --use_cpu
PARAMETER ...
Namespace(use_cpu=True, gpu='0', batch_size=24, model='pointnet2_cls_ssg', num_category=40, epoch=200,
    learning_rate=0.001, num_point=1024, optimizer='Adam', log_dir='pointnet2_modelnet40', decay_rate=0.0001,
    use_normals=False, process_data=False, use_uniform_sample=False)
Load dataset ...
Traceback (most recent call last):
  File "/tmp/pointnet-upstream/train_classification.py", line 232, in <module>
    main(args)
  File "/tmp/pointnet-upstream/train_classification.py", line 121, in main
    train_dataset = ModelNetDataLoader(root=data_path, args=args, split='train', process_data=args.process_data)
                    ^^^
```

APPENDIX D / LOCAL EVIDENCE

VoxelCNN and MeshCNN smoke outputs

These images validate tensor and edge-network code paths on generated geometry only.

Mesh-to-voxel and 3D CNN forward pass

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python run_voxel_demo.py --grid-size 32
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
Section 7.3 | mesh-to-voxel + CNN forward pass
Mesh fixture: data/ModelNet10/chair/train/chair_0001.off (synthetic closed cube, 12 triangles)
Voxel grid: (32, 32, 32); occupied cells: 24389/32768 (74.43%)
Tensor shape (B,C,D,H,W): (1, 1, 32, 32, 32); dtype=torch.float32
Device: cpu; CUDA available: False
Output logits shape: (1, 10) (10 classes)
Predicted class index (untrained demo head): 7
STATUS: PASS — conversion and forward shape checked; synthetic fixture only, no ModelNet10 accuracy claimed.
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme

Figure D1. Synthetic-cube 32^3 voxelization and untrained 10-class CNN forward pass.

MeshCNN edge-network smoke run

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python run_meshcnn_smoke.py --meshcnn-root /tmp/meshcnn-upstream
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
Section 7.4 | upstream MeshCNN CPU smoke test
MeshCNN source: /tmp/meshcnn-upstream
Fixture: data/meshcnn_fixture/icosphere.obj (generated trimesh icosphere; not SHREC)
Topology: vertices=42, triangles=80, edges=120
Edge feature tensor: (5, 120) (dihedral angle, opposite angles, ratios)
Input tensor: (1, 5, 120, 1); logits: (1, 30) (30-way SHREC-like head only)
Single synthetic-label cross-entropy: 3.177232; backward/Adam step: PASS
STATUS: PASS — upstream MeshConv/connectivity path exercised; no SHREC training or accuracy claimed.
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme

Figure D2. Synthetic icosphere MeshCNN forward/optimizer smoke; not SHREC'16.

APPENDIX E / LOCAL EVIDENCE

Representation conversions

The terminal transcript below accompanies the locally generated conversion figure in Section 7.5.

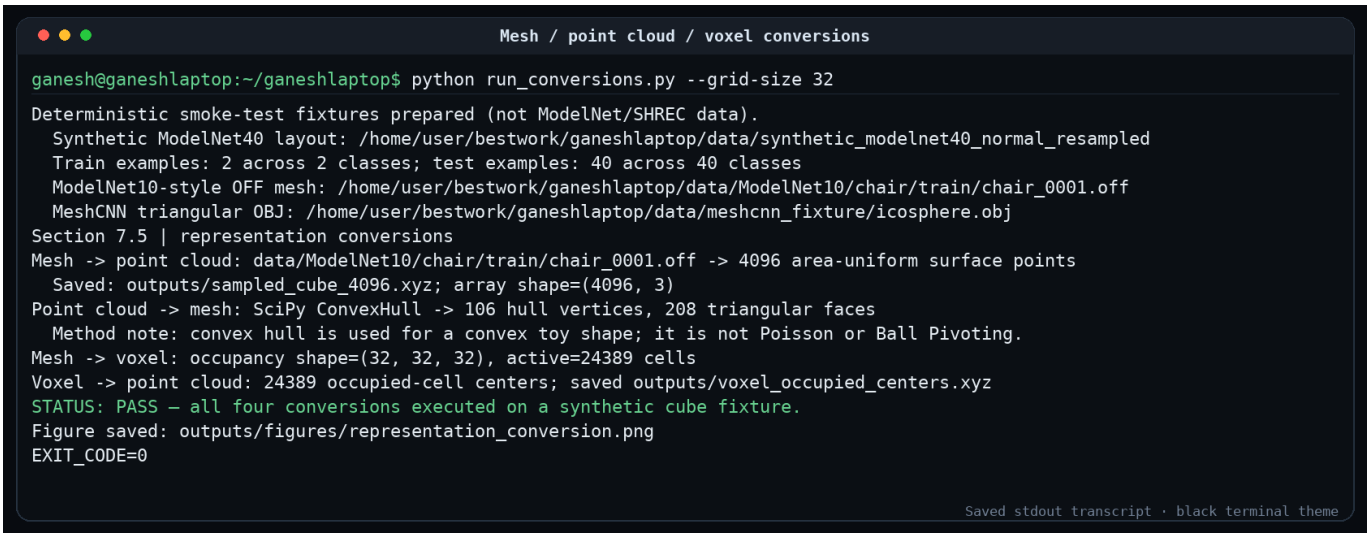


Figure E1. Local conversion output: 4,096 surface samples, convex hull and occupancy-center extraction on a synthetic cube.

Local conversion counts: 4,096 sampled surface points; 106 convex-hull vertices; 208 hull faces; 24,389 occupied voxels and the same number of occupied-cell centers. The generated shape, occupancy fill and convex-hull method differ from the single-chair Poisson / Ball Pivoting Chamfer study attributed to [9].

Submission identity

J GANESH KUMAR REDDY • 23BDS024 | Every page of this revised PDF carries this identity in its running header/footer.